CODE FOR INTERFACE TO LAUNCH

```
import React, { useState } from 'react';
import { Send, Bot, Settings, X, Plus, Globe, Loader } from 'lucide-react';

const AIWebToolsChatbotLauncher = () => {
 const [bots, setBots] = useState([]);
 const [currentBot, setCurrentBot] = useState(null);
 const [showNewBotForm, setShowNewBotForm] = useState(false);
 const [newBot, setNewBot] = useState({
  name: '',
  apiKey: '',
  model: 'gpt-3.5-turbo',
  instructions: '',
  operationalInstructions: '',
  webEnabled: false,
 });
 const [messages, setMessages] = useState([]);
 const [userInput, setUserInput] = useState('');
 const [isLoading, setIsLoading] = useState(false);
 const [error, setError] = useState('');

 const handleCreateBot = () => {
  console.log("Creating new bot:", newBot);
  if (newBot.name && newBot.apiKey && newBot.instructions) {
   setBots(prevBots => [...prevBots, { ...newBot, id: Date.now() }]);
   setShowNewBotForm(false);
   setNewBot({
    name: '',
    apiKey: '',
    model: 'gpt-3.5-turbo',
    instructions: '',
    operationalInstructions: '',
    webEnabled: false,
   });
  } else {
   setError('Please fill in all required fields (Name, API Key, and Instructions).');
```

```
  }
};

const handleLaunchBot = (bot) => {
  console.log("Launching bot:", bot);
  setCurrentBot(bot);
  setMessages([{ role: 'system', content: bot.instructions }]);
  setError('');
};

const handleSendMessage = async () => {
  if (userInput.trim()) {
    const newMessages = [...messages, { role: 'user', content: userInput }];
    setMessages(newMessages);
    setUserInput('');
    setIsLoading(true);
    setError('');

    console.log("Sending message to OpenAI API");
    try {
      const response = await fetch('https://api.openai.com/v1/chat/completions', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
          'Authorization': `Bearer ${currentBot.apiKey}`
        },
        body: JSON.stringify({
          model: currentBot.model,
          messages: [
            { role: 'system', content: currentBot.instructions + '\n' + currentBot.operationalInstructions },
            ...newMessages
          ]
        })
      });

      const data = await response.json();

      if (!response.ok) {
        throw new Error(data.error?.message || 'An error occurred while fetching the response.');
```

```
      }

      console.log("Received response from OpenAI API:", data);
      setMessages([...newMessages, data.choices[0].message]);
    } catch (err) {
      console.error("Error in API call:", err);
      setError(err.message);
    } finally {
      setIsLoading(false);
    }
  }
};

console.log("Rendering component. Current state:", { bots, currentBot, messages, isLoading, error });

return (
  <div className="max-w-4xl mx-auto p-4 bg-gray-100 rounded-lg shadow-md">
    <div className="flex justify-between items-center mb-6">
      <h1 className="text-3xl font-bold text-blue-600">AI WEB TOOLS GPT CHATBOT LAUNCHER</h1>
      <div className="w-16 h-16 bg-blue-500 rounded-full flex items-center justify-center">
        <svg viewBox="0 0 100 100" className="w-12 h-12 text-white">
          <text x="50" y="50" fontSize="40" fill="currentColor" textAnchor="middle"
dominantBaseline="central">AWT</text>
        </svg>
      </div>
    </div>
    <div className="text-sm text-gray-500 mb-4">by AIWEBTOOLS.AI</div>


    {error && <div className="bg-red-100 border border-red-400 text-red-700 px-4 py-3 rounded relative mb-4"
role="alert">{error}</div>}

    {!currentBot ? (
      <div>
        <div className="mb-4 flex justify-between items-center">
          <h3 className="text-xl font-semibold">Your Bots</h3>
          <button
            onClick={() => setShowNewBotForm(true)}
            className="bg-green-500 text-white p-2 rounded hover:bg-green-600 flex items-center"
          >
```

```
    <Plus size={20} className="mr-1" /> New Bot
  </button>
</div>
{bots.map(bot => (
  <div key={bot.id} className="mb-2 p-2 bg-white rounded shadow flex justify-between items-center">
    <span>{bot.name} ({bot.model})</span>
    <button
      onClick={() => handleLaunchBot(bot)}
      className="bg-blue-500 text-white p-2 rounded hover:bg-blue-600"
    >
      Launch
    </button>
  </div>
))}
{showNewBotForm && (
  <div className="mt-4 p-4 bg-white rounded shadow">
    <h3 className="text-xl font-semibold mb-2">Create New Bot</h3>
    <input
      type="text"
      value={newBot.name}
      onChange={(e) => setNewBot({...newBot, name: e.target.value})}
      className="w-full p-2 mb-2 border rounded"
      placeholder="Bot Name"
    />
    <input
      type="password"
      value={newBot.apiKey}
      onChange={(e) => setNewBot({...newBot, apiKey: e.target.value})}
      className="w-full p-2 mb-2 border rounded"
      placeholder="API Key"
    />
    <select
      value={newBot.model}
      onChange={(e) => setNewBot({...newBot, model: e.target.value})}
      className="w-full p-2 mb-2 border rounded"
```

```
>
  <option value="gpt-3.5-turbo">GPT-3.5-Turbo</option>
  <option value="gpt-4">GPT-4</option>
  <option value="gpt-4-32k">GPT-4-32K</option>
</select>
<textarea
  value={newBot.instructions}
  onChange={(e) => setNewBot({...newBot, instructions: e.target.value})}
  className="w-full p-2 mb-2 border rounded"
  rows="4"
  placeholder="Basic Instructions (required)"
/>
<textarea
  value={newBot.operationalInstructions}
  onChange={(e) => setNewBot({...newBot, operationalInstructions: e.target.value})}
  className="w-full p-2 mb-2 border rounded"
  rows="6"
  placeholder="Operational Instructions (up to 8000 characters)"
  maxLength={8000}
/>
<div className="mb-2">
  <label className="flex items-center">
    <input
      type="checkbox"
      checked={newBot.webEnabled}
      onChange={(e) => setNewBot({...newBot, webEnabled: e.target.checked})}
      className="mr-2"
    />
    Enable Web Connectivity
  </label>
</div>
<button
  onClick={handleCreateBot}
  className="w-full bg-green-500 text-white p-2 rounded hover:bg-green-600"
>
```

```
            Create Bot
          </button>
        </div>
      )}
    </div>
  ) : (
    <div>
      <div className="flex justify-between items-center mb-4">
        <h2 className="text-2xl font-bold">{currentBot.name} ({currentBot.model})</h2>
        <div>
          {currentBot.webEnabled && <Globe size={24} className="inline-block mr-2 text-blue-500" />}
          <button onClick={() => setCurrentBot(null)} className="text-gray-500 hover:text-gray-700">
            <X size={24} />
          </button>
        </div>
      </div>
      <div className="bg-white p-4 rounded shadow mb-4 h-64 overflow-y-auto">
        {messages.map((msg, index) => (
          msg.role !== 'system' && (
            <div key={index} className={`mb-2 ${msg.role === 'user' ? 'text-right' : 'text-left'}`}>
              <span className={`inline-block p-2 rounded ${msg.role === 'user' ? 'bg-blue-100' :
'bg-gray-200'}`}>
                {msg.content}
              </span>
            </div>
          )
        ))}
        {isLoading && (
          <div className="flex justify-center items-center">
            <Loader className="animate-spin" />
          </div>
        )}
      </div>
      <div className="flex">
        <input
          type="text"
```

```
            value={userInput}
            onChange={(e) => setUserInput(e.target.value)}
            className="flex-grow p-2 border rounded-l"
            placeholder="Type your message..."
            onKeyPress={(e) => e.key === 'Enter' && handleSendMessage()}
          />
          <button
            onClick={handleSendMessage}
            className="bg-blue-500 text-white p-2 rounded-r hover:bg-blue-600"
            disabled={isLoading}
          >
            <Send size={20} />
          </button>
        </div>
      </div>
    )}
  </div>
 );
};

export default AIWebToolsChatbotLauncher;
```